



Reichman University

Efi Arazi School of Computer Science

M.Sc. Program in Machine Learning and Data Science

Final Project Report

Permuted Dogs versus Cats

Student: Roy Rubin **ID:** 201312907

Advisors: Alon Oring (Reichman University)

July 10, 2026

Abstract

This report evaluates binary Dogs-versus-Cats classification under fixed deterministic tile-wise image permutations. Each image from the Kaggle Dogs vs Cats dataset is divided into a square grid, its tiles are rearranged by a deterministic permutation, and the same permutation is applied to both training and validation images for a given run. The study is a computer-vision benchmark motivated by structured permutations; it is not a cryptographic security experiment.

Part 1 compares frozen ImageNet-1k-pretrained MobileNetV3-Small, DeiT-Tiny, and gMLP-S16 backbones with trained binary classification heads over one unpermuted baseline and deterministic 4×4 , 7×7 , and 10×10 grids, each paired with easy, medium, and hard presets. Part 1 also includes two additional control experiments: full fine-tuning of the pretrained binary-head models and zero-shot evaluation of each native ImageNet-1k head by comparing summed dog-class and cat-class probabilities. Part 2 evaluates configurable CNN-backbone strategies relative to the regular Part 1 reference baseline, including patch shuffle, regular image augmentation, mixed original/permuted training, a nonlinear MLP classification head, and two curriculum variants. Because the explicit Part 2 ablation runs use 30 epochs while the regular Part 1 reference uses 10 epochs, the Part 2 deltas are descriptive rather than isolated estimates of each intervention. Part 3 computes model-agnostic permutation hardness metrics and joins them to saved configurable-CNN validation results without training additional models.

The findings are limited to fixed deterministic permutations, one train/validation split, one main seed, frozen pretrained backbones, no independent test set, and exploratory hardness correlations over a small deterministic matrix. Best validation accuracy is optimistic because it selects the best epoch on the validation set. Within these limits, the results provide preliminary evidence that useful dog/cat cues survive the tested deterministic permutations, but they do not show that the models reconstruct, invert, transfer across, or explicitly learn the original image layout.

1 Introduction

Image classification models are typically evaluated on data whose spatial structure is intact. This report asks how much binary Dogs-versus-Cats classification performance survives when that structure is deliberately disrupted by a fixed tile-wise permutation. Each input image is split into an $N \times N$ grid, the tiles are rearranged by a deterministic mapping, and the classifier must still distinguish dogs from cats on a held-out validation set transformed by the same target permutation.

The main research question is: *under a fixed supervised train/validation split, how do grid size, deterministic permutation preset, and pretrained visual representation affect validation accuracy after tile-wise image permutation?* The report also asks two secondary questions. First, which configurable-CNN training or head modifications improve validation accuracy relative to the regular Part 1 reference baseline? Second, do deterministic, model-agnostic permutation hardness metrics correlate with the observed configurable-CNN best validation accuracy across the final experiment matrix?

The task is motivated by permuted-puzzle problems and cryptographic hardness [2], but the cryptographic connection is motivational rather than directly tested. The present image setting is a relaxed empirical analogue: it uses natural images rather than graphs of functions, applies coarse tile-wise rather than pixel-wise permutations, and evaluates supervised classification rather than cryptographic recovery or security. The results should therefore be interpreted as evidence about this computer-vision testbed, not as evidence about cryptographic hardness.

The study contributes a controlled, reproducible empirical benchmark for analyzing

how fixed spatial rearrangements affect transfer-learning classifiers within this experimental matrix. The backbone comparison deliberately spans three commonly used visual-representation families: convolutional networks, transformer-based patch attention, and MLP-style token mixing.

- It defines a deterministic Dogs-versus-Cats tile-wise permutation matrix over one unpermuted baseline and 4×4 , 7×7 , and 10×10 grids, each crossed with easy, medium, and hard presets.
- It compares three frozen ImageNet-1k-pretrained representation families: convolutional MobileNetV3-Small, attention-based DeiT-Tiny, and gMLP-S16.
- It evaluates six configurable-CNN improvement strategies against the regular Part 1 reference baseline: patch shuffle, standard image augmentation, mixed original/permuted training, a nonlinear MLP head, and two curricula. Since the explicit Part 2 ablation runs use 30 epochs and the regular Part 1 reference uses 10 epochs, these deltas are descriptive.
- It quantifies each deterministic permutation with model-agnostic hardness metrics and relates those metrics to held-out validation accuracy.

The report is organized into three experimental components. Part 1 trains model baselines on the one-tile/no-permutation baseline and deterministic tile-wise permutations at 4×4 , 7×7 , and 10×10 resolution. Part 2 tests improvement strategies for the same permuted setting, including patch shuffle, regular image augmentation, mixed original/permuted training, a larger configurable-CNN classification head, and two curricula. Part 3 computes permutation hardness metrics and joins

them to validation accuracy so that image-independent permutation properties can be compared against model performance.

2 Related Works

2.1 Cryptographic and Permuted-Puzzle Motivation

The motivating proposal cites permuted puzzles as a source of cryptographic hardness, where fixed permutations can make structured objects difficult to recover or classify [2]. This report does not solve or evaluate a cryptographic task. Instead, it borrows the idea of structured permutation as a controlled way to disrupt spatial organization in images. The cryptographic motivation provides conceptual background, while the empirical claims are limited to supervised classification on the Dogs vs Cats benchmark under deterministic tile-wise permutations.

2.2 Spatial Ordering

Spatial ordering is a central source of information in images. Nearby pixels and patches usually belong to related object parts, textures, contours, or background regions, and many visual recognition methods exploit this structure either directly or through learned representations. Context-prediction and jigsaw-style work show that relative patch position and tile arrangement contain semantic signal [3, 12]; rotation prediction similarly demonstrates that image geometry can support representation learning [5]. The present report uses spatial reordering differently: the task is supervised Dogs-versus-Cats classification under a fixed deterministic tile-wise permutation, not reconstruction of the original image and not prediction of the permutation.

2.3 Convolutions

Convolutional neural networks are built around locality, weight sharing, and translation-related structure. These assumptions are well matched to ordinary natural images, where local textures, edges, and object parts often carry semantic information. MobileNetV3 is a compact convolutional family designed with efficient inverted residual blocks, squeeze-and-excitation, and architecture-search-informed components [10]. Tile-wise permutation disrupts many adjacency relations between neighboring image regions while preserving local content inside each tile, making a convolutional backbone a useful reference point for measuring how much local visual evidence survives the permutation.

2.4 Vision Transformers

Vision Transformers represent an image as a sequence of patch tokens and use self-attention to exchange information across the sequence [4]. DeiT extends this family with data-efficient ImageNet-scale training and distillation-aware design choices [15]. This patch-token view is relevant for tile-wise permutation because individual patches may remain informative even when global spatial arrangement is corrupted. At the same time, positional embeddings, patch size, attention depth, and pretraining all affect how a transformer responds to reordered image content.

2.5 MLPs and MLPs for Vision

Multilayer perceptrons are usually associated with channel-wise feature recombination, but vision MLP architectures also mix information across spatial positions. MLP-Mixer showed that patch-token features can be processed with alternating token-mixing and channel-mixing MLP layers [14]. gMLP uses

spatial gating to mix token information without self-attention and has been shown to work on both language and vision tasks [11]. This makes gMLP-S16 a useful non-convolutional, non-attention comparison point for testing whether patch-level token mixing can classify fixed tile-permuted images.

2.6 Robustness, Augmentation, and Curricula

The CNN-backbone ablations connect to broader regularization and robustness ideas in deep learning [7]. Patch shuffle and regular image augmentations introduce additional stochastic variation during training; such augmentation is a standard way to improve generalization when the training perturbations match useful invariances [13]. Mixed original/permuted training tests whether clean-layout examples help the classifier head fit cues that transfer to the permuted validation distribution. Curriculum learning variants test whether staged exposure to increasing corruption probability or increasing permutation difficulty improves optimization [1]. In this report, these methods are evaluated as practical interventions for a frozen pretrained configurable CNN backbone, not as general claims about robustness under all spatial corruptions.

3 Data Overview

3.1 Dataset Source and Image Sizes

The dataset is Kaggle’s Dogs vs Cats image dataset. The labeled portion contains 25,000 RGB JPEG images, with 12,500 cat images and 12,500 dog images. Original image dimensions are heterogeneous: in the local dataset copy, widths range from 42 to 1050 pixels and heights range from 32 to 768

pixels, with 8,513 unique width-height pairs. The most common sizes are approximately 500×374 and 499×375 , but the preprocessing pipeline resizes images before tiling so that all model inputs in a run have a consistent square resolution.

3.2 Supervised Split

The supervised split is stratified with seed 42 and an 80/20 train/validation ratio. This gives 20,000 training images and 5,000 validation images. Both splits are exactly class-balanced: the training split contains 10,000 cats and 10,000 dogs, and the validation split contains 2,500 cats and 2,500 dogs. The validation split is held out from optimization and is not modified by training-only augmentations such as patch shuffle, regular augmentation, original/permuted sampling, or curriculum-stage sampling. Table 10 provides the detailed Part 1 reuse of this split in Section 11.

3.3 Image Resizing and Model Input

All images are converted to RGB and resized to a square model input. The standardized timm input resolution is 224 pixels. When a requested tile grid does not divide 224 evenly, preprocessing first resizes to the smallest square size divisible by the grid side length, applies the tile rearrangement on that square tensor, and then center-crops back to 224. For example, a 10×10 grid uses a temporary 230×230 tile-compatible resize because 230 is the smallest multiple of 10 above 224. The final model tensor is normalized with ImageNet mean and standard deviation, matching the pretrained checkpoints used in the experiments.

3.4 Tile-Permutation Matrix

For the unpermuted setting, each image is processed as a normal dog or cat image and used as the benchmark for post-permutation performance. For the permuted setting, each image is divided into a square tile grid and the same deterministic tile-wise permutation is applied across images for a run. The final grid settings are one tile/no permutation, 16 tiles (4×4), 49 tiles (7×7), and 100 tiles (10×10). The one-tile/no-permutation baseline is repeated under the easy, medium, and hard labels only so that each part follows the same four grid settings crossed with three difficulty labels. Aggregate summaries should account for the fact that these three records correspond to the same unpermuted condition rather than three distinct permutations.

The easy, medium, and hard presets provide three controlled levels of spatial disruption at each nontrivial grid size. Easy performs a small number of local swaps and preserves most adjacency structure. Medium increases the fraction of swapped tiles and adds row and column shifts. Hard combines the largest swap fraction, broader swap distance, stronger shifts, and a global reversal. These presets make it possible to compare models not only as the number of tiles increases, but also as the deterministic transformation becomes more disruptive at a fixed grid size. The results reported below describe this specific experimental design.

4 Methodology

The experimental work is organized into three assignment parts. Part 1 establishes baselines by training on the unpermuted task and on fixed deterministic tile-wise permutations. Part 2 reuses the Part 1 configurable-CNN baseline and trains improvement ablations on the same task family. Part 3 computes

permutation descriptors and correlates them with the validation accuracy observed for the corresponding deterministic records.

Part 1 compares MobileNetV3-Small, DeiT-Tiny, and gMLP-S16. Part 2 and Part 3 use MobileNetV3-Small for the configurable-CNN experiments. Aggregation reports mean final validation accuracy, mean best validation accuracy, standard deviation across the three deterministic permutation labels at each tile count, and the number of runs. Validation accuracy is the main evaluation metric because the held-out validation set is class-balanced. Best validation accuracy is useful for comparing training behavior across runs, but it can be optimistic as a final performance estimate because it selects the best epoch using the validation set.

4.1 Models Used in the Baseline Comparison

Part 1 is designed as a family-level comparison among a CNN, a vision transformer, and an MLP-style vision model while keeping the pretraining source aligned. MobileNetV3-Small is used as the convolutional baseline because it is compact and efficient while still exposing ImageNet-1k pretrained features [10]. DeiT-Tiny represents the transformer family in a small pretrained configuration, using patch embeddings, multi-head self-attention, and a classification token [15]. gMLP-S16 represents a non-convolutional, non-attention patch model in which spatial gating mixes information across patch tokens [11]. All three selected checkpoints are ImageNet-1k pretrained timm models, so the comparison avoids mixing different pretraining datasets when evaluating the three families.

The Part 1 control-experiment matrix is reported through three conditions: the main frozen-backbone pretrained binary-head set-

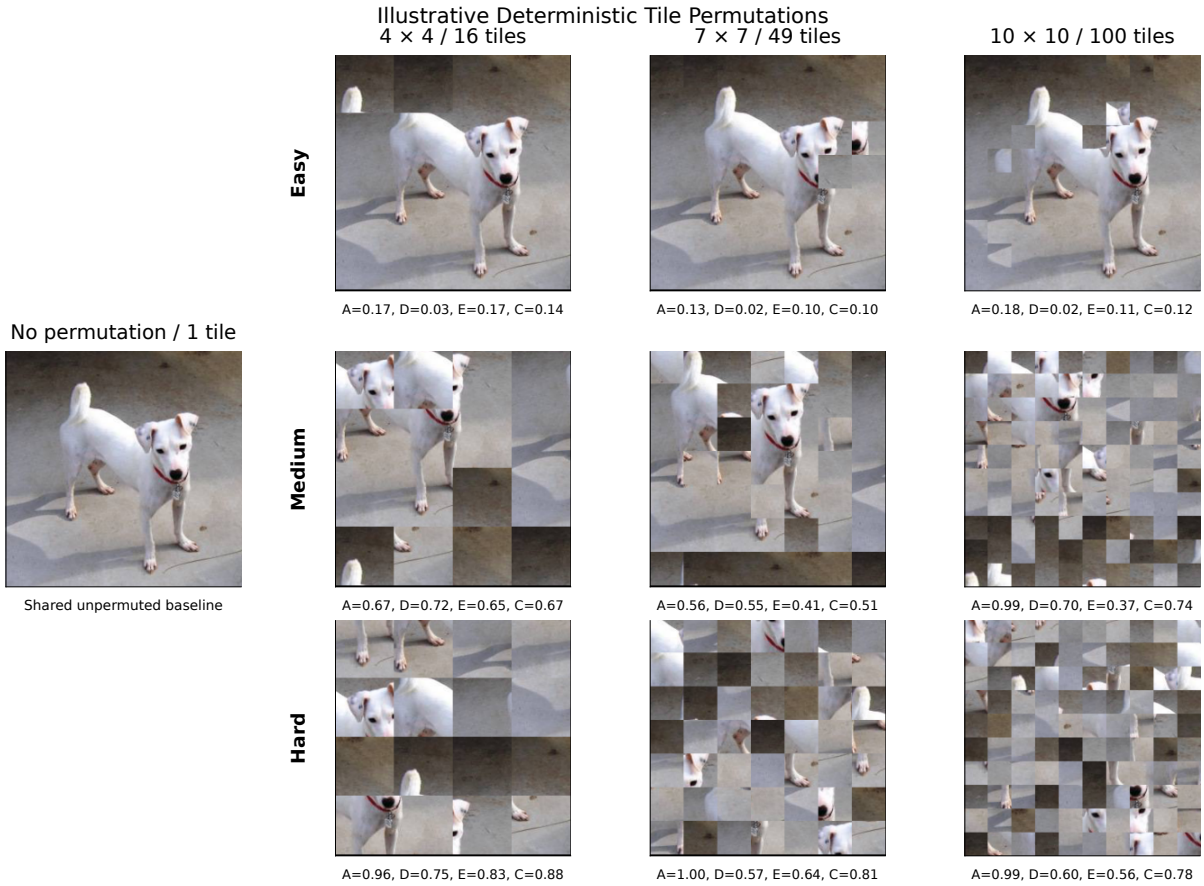


Figure 1: Illustrative validation image under the deterministic grid and difficulty settings used throughout the experiments. The first column shows one central unpermuted baseline image because the easy, medium, and hard one-tile records are visually identical. Rows correspond to the easy, medium, and hard presets for the nontrivial grids; columns correspond to the 4×4 , 7×7 , and 10×10 grids. Metric values are shown below each nontrivial grid image: adjacency destruction (A), global displacement (D), spatial entropy (E), and combined hardness (C). Figure 1 is a qualitative illustration of the deterministic transformations and is not evidence of model performance.

ting, an unfrozen pretrained binary-head control, and a zero-shot native ImageNet-1k-head control. The frozen condition is the primary baseline because it compares fixed representations plus trained binary heads rather than full end-to-end adaptation; the two controls test whether the conclusions change when the pretrained backbone is allowed to update or when no dog/cat binary training is performed.

4.2 Deterministic Permutation Functions

Every part uses one shared deterministic permutation construction with three parameter presets: easy, medium, and hard. This makes comparisons across Parts 1, 2, and 3 reproducible and interpretable, because a given difficulty label refers to the same construction wherever it appears.

The construction is parameterized by relative quantities: the fraction of tiles to swap, the maximum swap distance as a fraction of grid width, row and column shift fractions, and whether to apply a global reversal before the shifts and swaps. A global reversal means that the flattened row-major tile order is reversed, sending the first tile to the last position, the second tile to the second-to-last position, and so on before any subsequent shifts or swaps. Relative parameters are used because fixed operation counts are not comparable across grid sizes: two swaps affect 25% of the tiles in a 4×4 grid but only 4% of the tiles in a 10×10 grid. The easy preset therefore uses a small tile-count-scaled swap fraction and short local movement; medium increases the swap fraction and adds moderate row and column shifts; hard uses the largest swap fraction, broader movement radius, stronger shifts, and a global reversal. For the one-tile baseline, all three labels map to the unpermuted image; the labels are still

Preset	Swap frac.	Max dist. frac.	Row shift	Col shift	Reversal
Easy	0.05	0.15	0.00	0.00	No
Medium	0.14	0.35	0.20	0.20	No
Hard	0.28	0.80	0.45	0.45	Yes

Table 1: Deterministic permutation preset parameters, as defined in `tile_permutations.py`.

saved so the experiment matrix remains exactly 4 grid settings by 3 presets.

4.3 Architecture Comparison and Frozen Backbones

To compare architectural inductive biases, the Part 1 pipeline uses three pretrained feature extractors: MobileNetV3-Small [10], DeiT-Tiny [15], and gMLP-S16 [11]. All three are loaded from timm with ImageNet-1k-only pretrained weights. In the main Part 1 transfer-learning regime, the ImageNet-pretrained backbone is frozen and only the final binary classification head is optimized. This setting evaluates pretrained representations plus a small trained classification head under the tested permutations; it does not test full representation learning under permutation.

The three models differ in how they exchange information across the image. MobileNetV3-Small uses lightweight convolutions, giving it a local-neighborhood bias. DeiT-Tiny represents the image as patches and uses self-attention to let patches interact. gMLP-S16 also operates on image patches, but replaces attention with MLP layers and spatial gating for token mixing. The comparison is informative but not fully controlled: the models differ not only in inductive bias, but also in parameter count, patch/tokenization behavior, and feature geometry. Tables 7–9 summarize the model-family roles, parameter counts, checkpoint identifiers, and head-freezing behavior used in the executed comparison.

4.4 Optimization and Hyperparameters

The original Part 1 baseline and the explicit Part 2 ablation training runs use ImageNet-pretrained backbones with the feature extractor frozen and only the classification head trainable. The added unfrozen Part 1 control keeps the pretrained initialization and binary head but allows all model parameters to update. The zero-shot full-head control performs no gradient updates and reports validation accuracy from the native ImageNet-1k classifier. The updated Part 2 regular-baseline aggregate is loaded from the current regular Part 1 output rows and is used as the grid-level reference for ablation deltas.

For trained binary-head runs, the loss is cross-entropy over the two classes. The optimizer is AdamW with learning rate 3×10^{-4} and weight decay 1×10^{-4} . Runs use seed 42. In the executed Colab configuration, the batch size is 64, the dataloader uses 4 workers, and automatic mixed precision is enabled. The local configuration defaults to smaller smoke-test settings, so the reported results should be read as the Colab run matrix.

Part 1 trained binary-head runs train each model/permutation record for 10 epochs; the zero-shot full-head control uses 0 epochs. The explicit Part 2 ablation runs train for 30 epochs. In the curriculum ablations, those 30 epochs are distributed across stages as evenly as possible: corruption-probability curriculum uses four stages with probabilities 0.1, 0.3, 0.5, and 0.8, while permutation-difficulty curriculum starts from original images and progresses through 2×2 , 3×3 , 4×4 , and, when needed, the final target grid. Part 3 does not train a model; it computes deterministic permutation metrics and joins them to the saved configurable-CNN accuracies.

4.5 Preprocessing and Training-Only Augmentation

Section 3 describes the shared resizing, tile-compatible preprocessing, and ImageNet normalization used by the model inputs. Training-only perturbations are applied after the target deterministic run condition is defined. Regular augmentation uses random rotation up to 15 degrees, color jitter with brightness/contrast/saturation 0.2 and hue 0.05, resize, tensor conversion, and normalization. Patch shuffle uses a random within-image patch permutation during training only. Validation loaders do not apply these training-only transformations; they use the deterministic target permutation for the run.

4.6 Improvement Strategies

Part 2 evaluates training strategies against the regular Part 1 configurable-CNN baseline, which defaults to MobileNetV3-Small. The main distinction is whether a strategy adds stochastic training views, mixes clean and permuted inputs, changes the classifier head, or changes the order in which the model sees easy and hard inputs. In all cases, validation is performed on the held-out validation images with the target permutation and without training-only perturbations.

For the counts below, let B be the configured batch size and let $N_{\text{train}} = 20,000$ be the number of training images. The ablations do not append files to the dataset. Instead, they replace or resample the ordinary training view during the forward pass, so the stored increase is zero even when every processed example is augmented or curriculum-scheduled.

4.6.1 Regular Part 1 Baseline

Main idea. This is the reference point for Part 2. The configured CNN backbone is trained normally on the target permutation, with no extra augmentation, no curriculum, and the standard loss.

Reason. The baseline is necessary because every ablation must be judged against ordinary training on the same task. Without it, an improvement strategy might look useful simply because the target permutation is easier than another run.

Deeper dive. The baseline uses the same train and validation split as Part 1. For a permuted run, both training and validation images are transformed by the same fixed tile permutation for that run. It has one training stage and does not increase the dataset size. The number of added augmented images is 0 per batch and 0 per epoch; the number of processed training views remains B per batch and 20,000 per epoch.

4.6.2 Patch Shuffle

Main idea. Patch shuffle randomly rearranges smaller patches during training. The model therefore sees extra local spatial disruptions in addition to the fixed tile permutation.

Reason. This is relevant because the main task already tests whether class information survives spatial reordering. If extra random patch movement helps, it would suggest that tolerance to local layout disruption is useful for the tested permuted Dogs versus Cats setting.

Deeper dive. The target permutation is still the run-level task definition, but each training batch can also receive a random within-image patch permutation sampled independently during training. In the implementa-

tion the shuffle probability is 1.0, so a batch of size B contains B shuffled views and an epoch contains 20,000 shuffled views. These views are not appended to the dataset; they replace the current batch examples. The validation loader does not use patch shuffle, so reported accuracy measures the fixed target validation permutation.

4.6.3 Regular Augmentations

Main idea. Regular augmentations apply standard image-level stochastic transformations during training while keeping the target validation permutation fixed.

Reason. This tests whether generic image augmentation helps the frozen CNN head fit dog-versus-cat cues that transfer to the target tile permutation, or whether the extra variation is mismatched to the spatial corruption used at validation time.

Deeper dive. The training set remains the same 20,000 images. For each processed example, a stochastic image-level transform is sampled during training after the target permutation is defined. The strategy has one stage and trains for 30 epochs in the saved Part 2 ablation matrix. It stores no additional images; it produces one stochastic view per processed training example. Validation uses the deterministic target permutation without training-only augmentation.

4.6.4 Mixed Original/Permuted Training

Main idea. Mixed original/permuted training exposes the model to clean images and target-permuted images in the same run.

Reason. If clean images preserve class structure that is useful for the permuted task, mixing them into training may stabilize the classifier. If the clean examples pull the head

away from the target validation distribution, the strategy may hurt.

Deeper dive. The ablation samples each training view as original with probability $p_{\text{original}} = 0.5$ and target-permuted otherwise. This gives an expected 10,000 clean views and 10,000 permuted views per epoch. The validation loader remains fully target-permuted, so the reported accuracy measures transfer from the mixed training distribution to the target permutation.

4.6.5 CNN MLP Head

Main idea. This ablation keeps the frozen configurable CNN backbone but replaces the linear classifier with a small MLP classification head.

Reason. The frozen backbone may still extract useful ImageNet features from permuted tiles, but the mapping from those features to dog/cat labels may be less linearly separable after spatial disruption. A small MLP head tests whether additional trainable capacity at the classifier stage improves the target task.

Deeper dive. The data, target permutations, validation loader, and 30-epoch Part 2 ablation schedule match the other explicit Part 2 improvement runs. The only intentional change is the classifier head, so improvements in this ablation can be interpreted as preliminary evidence that head capacity matters under the frozen-backbone setup.

4.6.6 Corruption-Probability Curriculum

Main idea. The model starts with mostly intact images and gradually sees the target permutation more often. The corruption probability increases across four stages.

Reason. Jumping immediately into a hard permutation may make optimization unstable. This curriculum tests whether gradual exposure lets the frozen-backbone classifier first fit dog-versus-cat cues and then adapt those cues to more frequent spatial corruption.

Deeper dive. Each stage uses the same 20,000 training images and target tile permutation, but changes the probability that the permutation is applied. The stages use probabilities 0.1, 0.3, 0.5, and 0.8, so the expected number of permuted views per stage epoch is 2,000, 6,000, 10,000, and 16,000 respectively. In a batch of size B , the expected number of permuted views is the stage probability times B . This does not store a larger dataset; it changes the sampled training view. There is no separate validation set for each stage; validation always uses the fixed target validation loader. Curriculum runs train for 30 epochs in the final saved results.

4.6.7 Permutation-Difficulty Curriculum

Main idea. The model starts on easier spatial structure and moves toward the final harder permutation. Instead of changing probability, it changes the tile grid difficulty across stages.

Reason. Smaller or easier permutations preserve more of the original image structure. This experiment tests whether a model can learn useful class cues under mild disruption before facing the final, harder tile rearrangement.

Deeper dive. Each stage uses the same 20,000 training images but changes the tile grid used for training. The schedule begins with original images and then progresses through 2×2 , 3×3 , and 4×4 permutations; for 7×7 and 10×10 target runs, it

adds a final target-grid stage. Stage permutations use the same deterministic difficulty label as the target record. Each stage processes B scheduled-difficulty views per batch and 20,000 per stage epoch. The stored dataset size does not increase, validation remains the held-out loader for the final target run, and curriculum runs train for 30 epochs in the final saved results.

4.7 Tile Permutation Hardness Metrics

We quantify the difficulty induced by a tile permutation using three complementary, model-agnostic axes and a fourth combined score. Let the image be divided into an $N \times N$ grid. For each source tile i , let (r_i, c_i) denote its original row and column, and let (r'_i, c'_i) denote its destination after applying the permutation.

4.7.1 Adjacency Destruction Hardness

Main idea. This metric asks how much of the original neighborhood structure survives after the tile permutation. If tiles that were next to each other are no longer neighbors, the image becomes harder for a convolutional model that relies on local structure.

For every original rightward and downward neighboring relation, we test whether the same two source tiles remain adjacent in the same direction after permutation. The number of such canonical adjacency relations is

$$M = 2N(N - 1).$$

If $A_{\text{preserved}}$ is the number of preserved directed adjacencies, adjacency destruction is

$$A = 1 - \frac{A_{\text{preserved}}}{M}.$$

A low value means that much of the original local structure is preserved. A high value

means that neighboring tile relations were mostly destroyed.

4.7.2 Global Tile Displacement

Main idea. This metric measures how far tiles move from their original positions. A permutation that only nudges tiles nearby should preserve more global shape than one that sends tiles across the image.

For each tile, we compute its Manhattan displacement in grid coordinates:

$$\delta_i = |r'_i - r_i| + |c'_i - c_i|.$$

The normalized global displacement is

$$D = \frac{\sum_i \delta_i}{D_{\max}},$$

where $D_{\max}$ is the maximum possible total Manhattan displacement for an $N \times N$ grid. A low value indicates that tiles remain close to their original locations. A high value indicates large-scale spatial relocation.

4.7.3 Spatial Permutation Entropy

Main idea. This metric asks whether tile movement follows a simple pattern or many different movement patterns. More varied movement directions and distances mean the permutation is less orderly.

For each tile, we compute

$$\Delta_i = (r'_i - r_i, c'_i - c_i)$$

and assign it to a movement bin

$$b_i = (\text{direction}(\Delta_i), \text{distanceTier}(\Delta_i)).$$

The direction is one of

$\{\text{stationary}, N, S, E, W, NE, NW, SE, SW\}$.

For non-stationary tiles, the normalized Manhattan distance is

$$d_i = \frac{|r'_i - r_i| + |c'_i - c_i|}{2(N - 1)}.$$

Metric	Dimension	What it captures
Adjacency destruction hardness	Local topology	Are original neighbors still neighbors?
Global tile displacement	Global geometry	How far did tiles move from their original positions?
Spatial permutation entropy	Permutation disorder	How diverse or irregular are tile movement patterns?
Combined hardness score	Aggregated hardness	Weighted summary of the three normalized axes.

Table 2: Tile permutation hardness metrics used in the analysis.

The distance tier is near when $0 < d_i \leq 1/3$, medium when $1/3 < d_i \leq 2/3$, and far when $2/3 < d_i \leq 1$. Stationary tiles use a single stationary bin. If p_b is the empirical probability of movement bin b , the normalized entropy is

$$E = \frac{-\sum_b p_b \log p_b}{\log(N^2)}.$$

A low value means that movement is structured or concentrated in a small number of movement patterns. A high value means that the permutation contains diverse movement directions and distances, making it more disorderly.

4.7.4 Combined Hardness Score

Main idea. The combined score summarizes the component metrics into one number so permutations can be ranked by an overall notion of difficulty.

The final score is a convex combination of the three normalized components:

$$C = w_a A + w_d D + w_e E,$$

with default weights

$$w_a = 0.5, \quad w_e = 0.3, \quad w_d = 0.2.$$

A low combined score represents an easier permutation with preserved structure, small movement, and low movement disorder. A high combined score represents a harder permutation across these complementary axes.

5 Additional Implementation Details

Table 4 collects the implementation details needed to reproduce the reported run matrix. These details are separated from the main methodology so that the scientific setup remains readable while still documenting the executed configuration.

5.1 Model Implementation Details

Tables 7–9 in Section 11 record the model-family summary, parameter counts, and concrete timm checkpoint identifiers used in the Part 1 comparison.

5.2 Training and Validation Sets by Experiment

All experiments use the same 20,000/5,000 split described in the data overview. Table 10 summarizes how that split is reused in the Part 1 runs.

The curriculum ablations in Part 2 use stage-specific training loaders, but they do not create separate held-out validation splits per stage. Across both curricula, validation remains the same 5,000-image held-out loader for the final target run. The saved explicit Part 2 ablation runs use 30 epochs, so curriculum comparisons are not explained by a shorter training schedule for the other improvement runs in the current matrix.

6 Evaluation Protocol

Validation accuracy is the main metric throughout the report because the held-out validation set is class-balanced, with equal numbers of cats and dogs. Final validation accuracy reports the value at the end of the configured training run. Best validation accuracy reports the maximum validation accuracy reached during training and is useful for comparing training behavior, but it can be optimistic as a final performance estimate because it selects the best epoch on the validation set.

For Part 3, the joined hardness table keeps the 12 deterministic records in the final configurable-CNN matrix for traceability. The reported correlation table uses the nine non-baseline permutation rows, so the clean repeated baseline does not dominate the association. The original 12-row view and one-baseline-row view are retained as sensitivity checks in the saved CSV outputs. The correlations should therefore be treated as exploratory descriptors of this controlled matrix rather than definitive statistical estimates. No p-values, confidence intervals, repeated-seed variability estimates, or independent test-set measurements are reported.

7 Results

The results are organized according to the three assignment parts. Section 7.1 reports the frozen-backbone comparison under deterministic tile permutations. Section 7.2 reports the configurable-CNN ablation matrix. Section 7.3 reports the model-agnostic hardness metrics and their relationship to validation accuracy.

All result statements in this section should be read as validation-set findings for one train/validation split, one main seed, frozen

pretrained backbones, and a limited deterministic permutation matrix with three pre-sets per grid size. They are not independent test-set estimates, and they do not quantify repeated-seed variability.

7.1 Part 1: Frozen-Backbone Baselines

Table 5 summarizes the frozen-backbone architecture comparison by tile count, and Figure 2 visualizes the main comparison together with the two saved controls. The notebook records the main frozen-backbone runs, the unfrozen control, and the zero-shot control with explicit condition labels so those results can be reported separately.

The main Part 1 analysis compares the three ImageNet-1k pretrained backbones under the same deterministic permutation matrix. If the hardest conditions remain far above chance, one plausible explanation is that tile permutation preserves local texture, color, object-part, and background cues, and ImageNet-trained classifiers are known to rely heavily on texture and other local cues [6, 9]. The notebook includes explicit checks of label order, validation balance, train/validation separation, and the hard-row calculation before drawing conclusions from the controls.

The frozen-backbone comparison shows a clear architecture effect. DeiT-Tiny and gMLP-S16 remain above 89% mean best validation accuracy even at 100 tiles, while MobileNetV3-Small drops to 80.36%. This does not prove that token models recover the original layout, but it does show that their pretrained features and binary heads retain more class-discriminative signal under this fixed matrix. The unfrozen control in Figure 2 is generally strong for the completed MobileNetV3-Small and gMLP-S16 records, while the zero-shot control falls more sharply

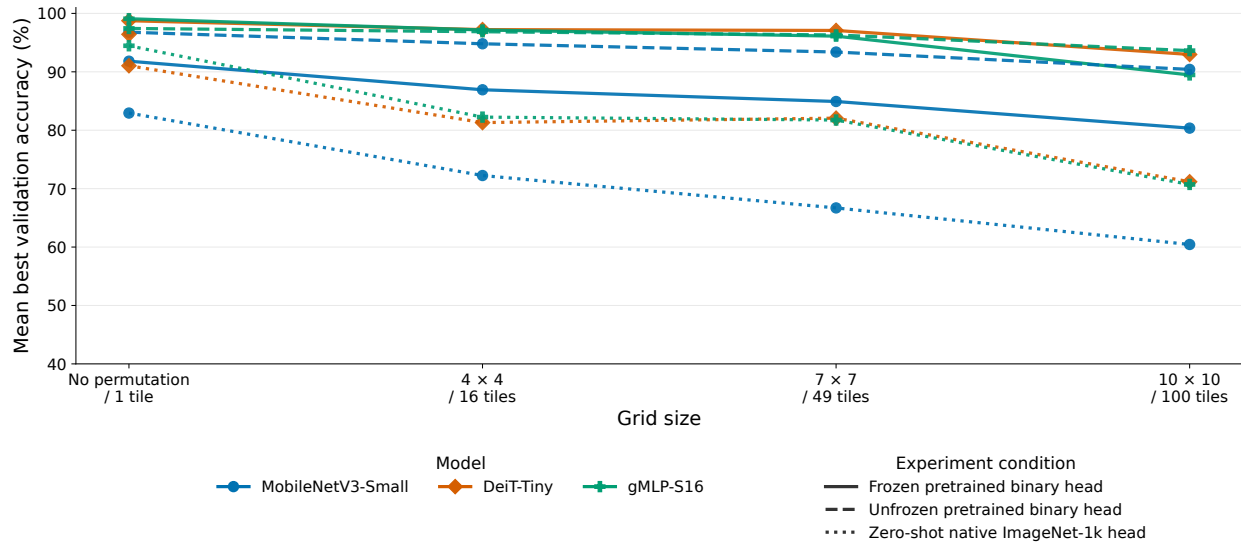


Figure 2: Part 1 mean best validation accuracy as a function of grid size for the three model families and three saved experiment conditions. Solid lines show the main frozen-backbone binary-head setting, dashed lines show the unfrozen pretrained binary-head controls, and dotted lines show the zero-shot native ImageNet-1k-head controls. Model color and marker shape are kept consistent across conditions. Control curves are included for comparison only. The saved DeiT-Tiny unfrozen control is incomplete and contains only the no-permutation record, so it is shown as a single dashed marker rather than a full curve. No confidence intervals are shown because repeated-seed estimates are not available.

as tile count and disruption increase. The zero-shot result is useful because it shows that the native ImageNet head already carries dog/cat signal, but supervised binary-head training is still important under the stronger permutations. Figure 3 in Section 11 provides the decomposed views corresponding to the combined comparison.

7.2 Part 2: Configurable-CNN Improvement Strategies

Table 6 reports the MobileNetV3-Small CNN ablation summary from the updated Part 2 aggregate CSV. The six explicit ablation rows each summarize 12 deterministic records; the regular baseline row summarizes the current regular-Part-1 aggregate used as the grid-level reference. Figure 4 shows each strategy’s delta from the regular Part 1 reference baseline in two complementary views.

The Part 2 matrix is designed to compare each ablation against the regular Part 1 reference at the same grid size and deterministic difficulty label. In the updated aggregate summary, the CNN MLP head has the largest mean best-validation improvement over the grid-level baseline (+8.49 percentage points), followed by the two curriculum variants and regular augmentations. Patch shuffle has the smallest mean improvement and is negative for the 100-tile aggregate, suggesting that additional random local shuffling is not uniformly helpful under the final fixed validation permutations. The explicit Part 2 ablations train for 30 epochs while the regular Part 1 reference baseline trains for 10 epochs, so the reported deltas are descriptive and should not be interpreted as isolated causal effects of the ablation alone.

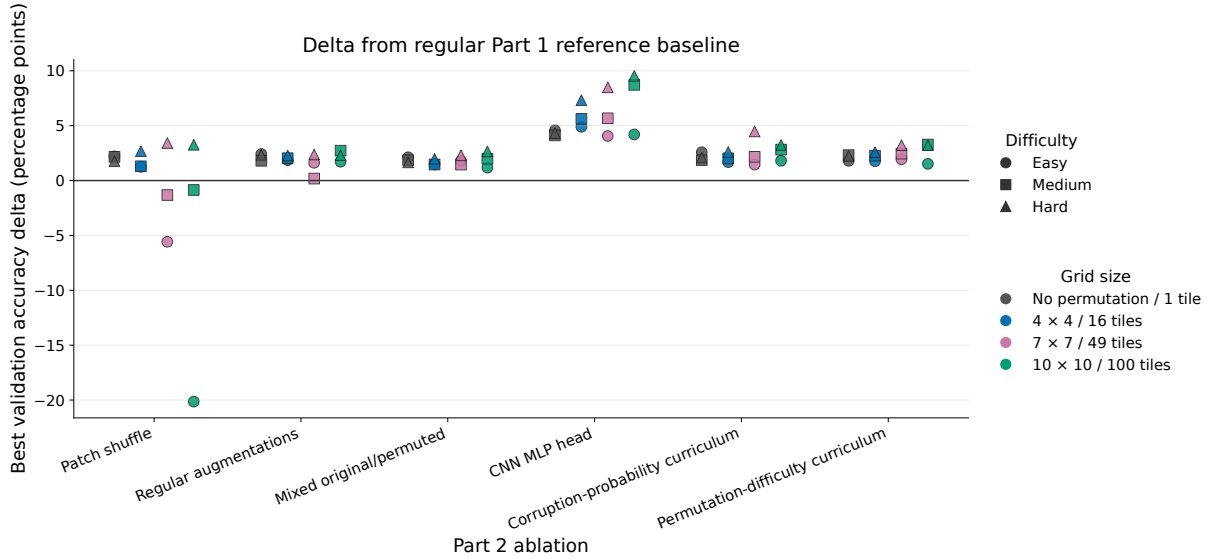
Hardness metric	Pearson	Spearman
Global displacement	-0.7093	-0.6167
Adjacency destruction	-0.9100	-0.9372
Spatial entropy	-0.6116	-0.6333
Combined hardness	-0.8331	-0.8667

Table 3: Part 3 exploratory correlations between MobileNetV3-Small best validation accuracy and permutation metrics over the nine non-baseline permutation records ($N = 9$ for each row). Negative values indicate that larger hardness scores are associated with lower validation accuracy in this matrix.

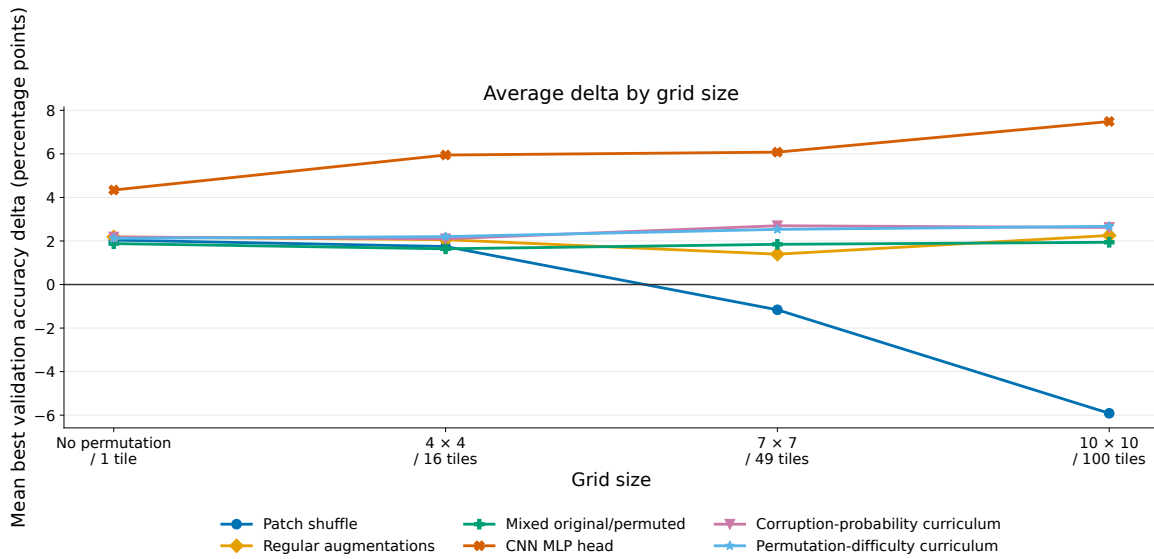
7.3 Part 3: Hardness Metrics

Table 3 reports exploratory correlations between deterministic hardness metrics and MobileNetV3-Small best validation accuracy under the nine non-baseline permutation records. Figure 1 shows the corresponding image-level examples, and Figure 8 in the Additional Information section visualizes all four hardness metrics by grid size and deterministic difficulty label.

Part 3 provides a ranking lens for the deterministic permutations. In the nine-row non-baseline scope, all four hardness metrics were negatively correlated with MobileNetV3-Small best validation accuracy. Adjacency destruction had the largest-magnitude single-metric association, with Pearson correlation -0.9100 and Spearman correlation -0.9372. The retained 12-row full scope and one-baseline-row scope give similar but baseline-weighted sensitivity views in the saved correlation CSV.



(a) Deltas grouped by Part 2 ablation strategy. Marker shape distinguishes difficulty; color and horizontal offsets distinguish grid size.



(b) Deltas grouped by grid size, with one line per Part 2 strategy. Values are averaged over the easy, medium, and hard labels for readability.

Figure 4: Part 2 best-validation-accuracy deltas from the regular Part 1 reference baseline. Positive values indicate higher best validation accuracy than the corresponding regular Part 1 reference. Deltas are relative to the regular Part 1 reference baseline; explicit Part 2 ablations use 30 epochs; the regular Part 1 reference uses 10 epochs; therefore the deltas are descriptive and should not be interpreted as isolated causal effects.

8 Discussion

The experiments show that Dogs-versus-Cats classification remains possible after fixed deterministic tile permutations, but performance still depends on grid size, permutation preset, and representation family. DeiT-Tiny and gMLP-S16 preserve higher validation accuracy than MobileNetV3-Small under the strongest tested grids, suggesting that the frozen token-based representations expose more usable dog/cat signal in this protocol. This should not be read as evidence that the models reconstruct the original layout or solve permutation hardness; local texture, object-part, color, and background cues may remain sufficient for this dataset.

The Part 2 ablations indicate that head capacity and curriculum-style training are more useful than adding random patch shuffle alone. The CNN MLP head gives the largest mean improvement over the regular Part 1 reference baseline, which is consistent with the idea that frozen convolutional features may remain informative but need a more flexible classifier head after spatial disruption. Patch shuffle is weaker and can hurt at larger grids, implying that extra random local corruption is not automatically aligned with the fixed validation permutations.

The hardness metrics provide a compact way to rank the tested permutations. Their negative correlations with MobileNetV3-Small accuracy, especially adjacency destruction, support the intuition that disrupting local neighborhoods is harmful for a frozen CNN. Because this analysis uses only nine non-baseline permutation records, it is best treated as hypothesis-generating rather than a stable statistical estimate.

9 Limitations

Several limitations constrain the strength and scope of the conclusions. First, no repeated training seeds are reported, so run-to-run variation is not quantified. Second, no independent labeled test set is used, so validation accuracy may overstate final generalization. Best validation accuracy is especially optimistic because it selects the best epoch on the validation set. Third, the architecture comparison is confounded by pretraining, parameter counts, patch sizes, tokenization behavior, and model-specific feature representations. Fourth, the frozen-backbone setup limits conclusions about how fully trainable models would adapt to tile-wise permutations.

The experimental matrix is also limited. The deterministic matrix contains only three permutation presets per grid size, and the reported hardness-correlation analysis uses only nine non-baseline permutation records. The study does not test unseen permutations, transfer across permutations, cryptographic recovery, reconstruction, inversion, or explicit learning of the original layout. Full fine-tuning is included only as a Part 1 control rather than as the main training regime or a Part 2 improvement strategy. Finally, Dogs-versus-Cats may remain classifiable from local texture, object-part, or background cues even when global structure is disrupted. The report therefore does not directly distinguish learning or compensating for a fixed permutation from exploiting remaining class-discriminative cues.

10 Conclusion and Future Work

This report presents a reproducible Dogs-versus-Cats testbed for studying fixed tile-

wise image permutations. The implementation uses MobileNetV3-Small, DeiT-Tiny, and gMLP-S16 as the ImageNet-1k-only pre-trained backbone set. The experiment matrix is designed to test whether fixed permutations reduce validation accuracy without eliminating class-discriminative signal, and whether the effect differs between convolutional, transformer, and MLP-style vision backbones.

Within the scope of the experiments, these results suggest that pretrained visual representations can preserve useful dog/cat information even when global image layout is disrupted by the tested deterministic tile-wise permutations. They also suggest that local adjacency destruction may be a useful model-agnostic descriptor of permutation difficulty for frozen CNNs in this matrix. The results do not prove that models reconstruct, invert, or explicitly learn the permutation, and they do not establish general robustness to unseen permutations, transfer across permutations, or cryptographic hardness.

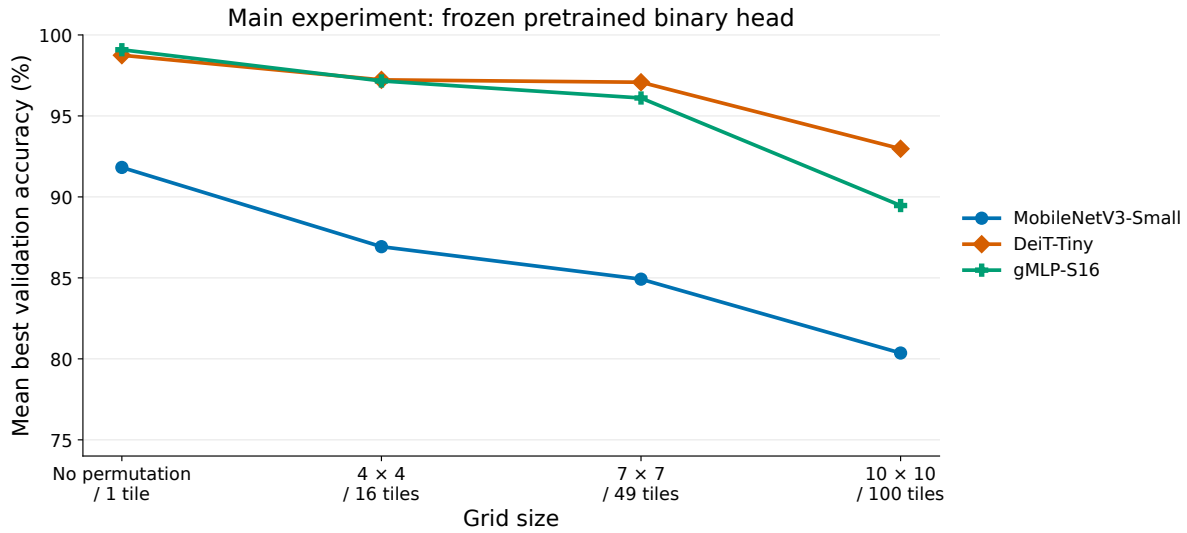
Future work should include repeated training seeds, independent test evaluation, full fine-tuning, more permutation samples per grid size, transfer tests between permutation functions, and experiments on datasets where global shape is more necessary for classification. Overall, the study provides preliminary empirical evidence and a reproducible framework for analyzing classification under deterministic spatial permutations, while showing that stronger statistical validation and transfer experiments are needed before drawing broader conclusions about permutation robustness.

References

- [1] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *International Conference on Machine Learning*, pages 41–48, 2009. <https://doi.org/10.1145/1553374.1553380>.
- [2] E. Boyle, J. Holmgren, and M. Weiss. Permuted puzzles and cryptographic hardness. In *Theory of Cryptography*, pages 465–493, 2019. https://doi.org/10.1007/978-3-030-36033-7_18.
- [3] C. Doersch, A. Gupta, and A. A. Efros. Unsupervised visual representation learning by context prediction. In *IEEE International Conference on Computer Vision*, pages 1422–1430, 2015. <https://arxiv.org/abs/1505.05192>.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. <https://arxiv.org/abs/2010.11929>.
- [5] S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *International Conference on Learning Representations*, 2018. <https://arxiv.org/abs/1803.07728>.
- [6] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*, 2019. <https://arxiv.org/abs/1811.12231>.
- [7] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. <https://www.deeplearningbook.org/>.
- [8] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. <https://doi.org/10.1109/CVPR.2016.90>.
- [9] K. L. Hermann, T. Chen, and S. Kornblith. The origins and prevalence of texture bias in convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2020. <https://proceedings.neurips.cc/paper/2020/hash/db5f9f42a7157abe65bb145000b5871a-Abstract.html>.
- [10] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam. Searching for MobileNetV3. In *IEEE/CVF International Conference on Computer Vision*, pages 1314–1324, 2019. <https://arxiv.org/abs/1905.02244>.
- [11] H. Liu, Z. Dai, D. R. So, and Q. V. Le. Pay attention to MLPs. In *Advances in Neural Information Processing Systems*, 2021. <https://arxiv.org/abs/2105.08050>.
- [12] M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving

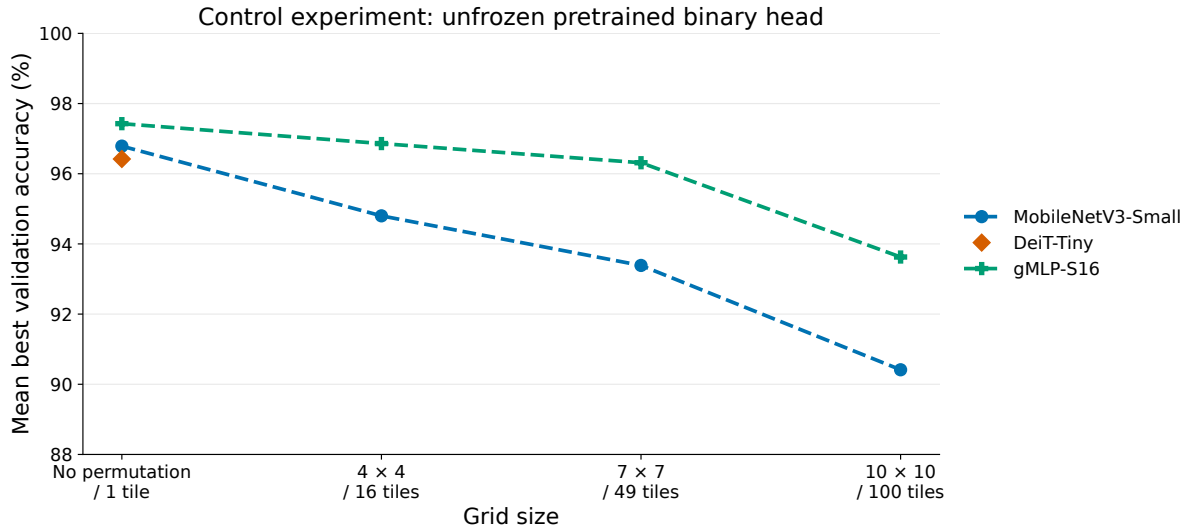
- jigsaw puzzles. In *European Conference on Computer Vision*, pages 69–84, 2016. <https://arxiv.org/abs/1603.09246>.
- [13] C. Shorten and T. M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019. <https://doi.org/10.1186/s40537-019-0197-0>.
- [14] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy. MLP-Mixer: An all-MLP architecture for vision. In *Advances in Neural Information Processing Systems*, 2021. <https://arxiv.org/abs/2105.01601>.
- [15] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers and distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021. <https://proceedings.mlr.press/v139/touvron21a.html>.

11 Additional Information



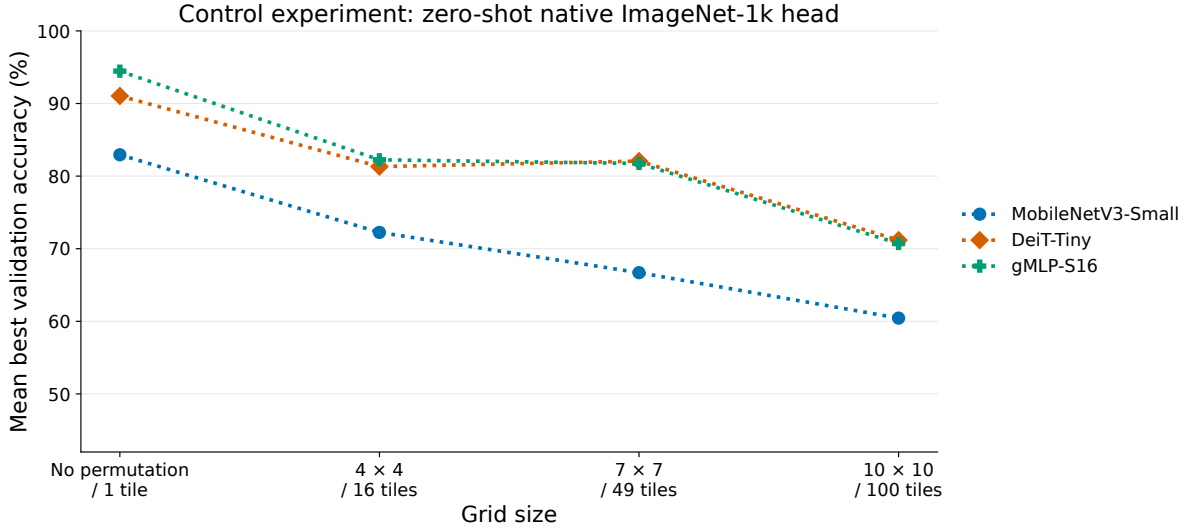
(a) Main frozen pretrained binary-head experiment.

Figure 3: Decomposed Part 1 views corresponding to the combined comparison in Figure 2. Validation accuracy is shown as mean best validation accuracy (%) using the same model colors, model markers, grid labels, and condition line styles as Figure 2. No confidence intervals are shown because repeated-seed estimates are not available.



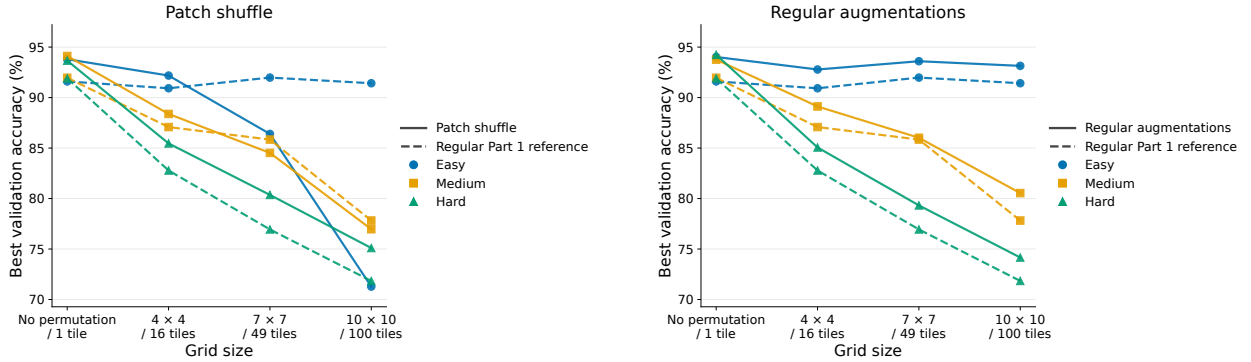
(b) Unfrozen pretrained binary-head control experiment.

Figure 3: Decomposed Part 1 views, continued. The saved DeiT-Tiny unfrozen control is incomplete and contains only the no-permutation record, so it is shown as a single marker rather than a complete curve.



(c) Zero-shot native ImageNet-1k-head control experiment.

Figure 3: Decomposed Part 1 views, continued. No binary training is performed in the zero-shot control. Dog probability is computed by summing ImageNet-1k class indices 151–268, and cat probability is computed by summing domestic-cat indices 281–285, matching the saved code. These controls are included for comparison only.



(a) Patch shuffle.

(b) Regular augmentations.

Figure 5: Intermediate Part 2 ablation views for training-time augmentation variants. Solid lines show the ablation; dashed lines show the regular Part 1 reference baseline. Axis ranges are shared across the intermediate ablation views to support visual comparison. These appendix figures are intermediate diagnostic views, not separate statistical tests.

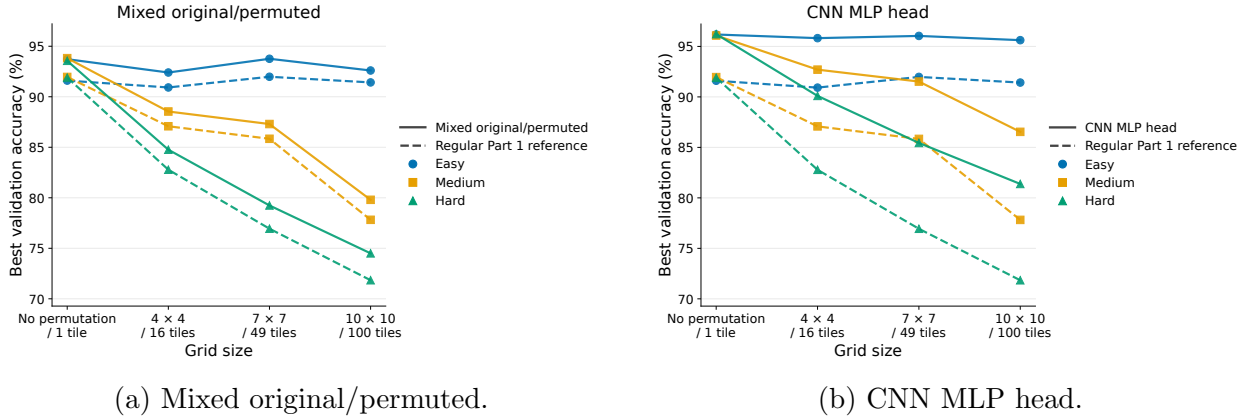


Figure 6: Intermediate Part 2 ablation views for mixed-input training and classifier-head capacity. Solid lines show the ablation; dashed lines show the regular Part 1 reference baseline. These appendix figures are intermediate diagnostic views, not separate statistical tests.

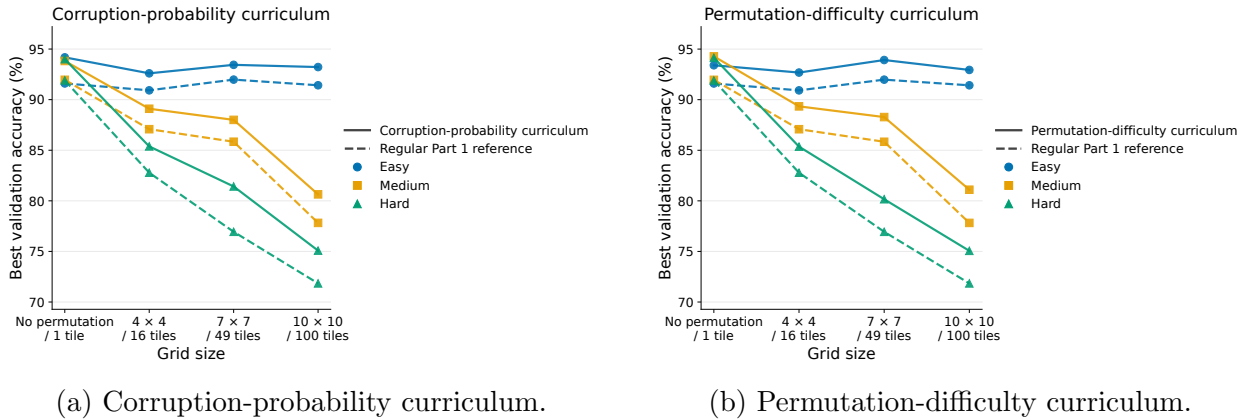


Figure 7: Intermediate Part 2 ablation views for curriculum variants. Solid lines show the ablation; dashed lines show the regular Part 1 reference baseline. The explicit curriculum ablations are trained for 30 epochs and are compared descriptively to the 10-epoch regular Part 1 reference. These appendix figures are intermediate diagnostic views, not separate statistical tests.

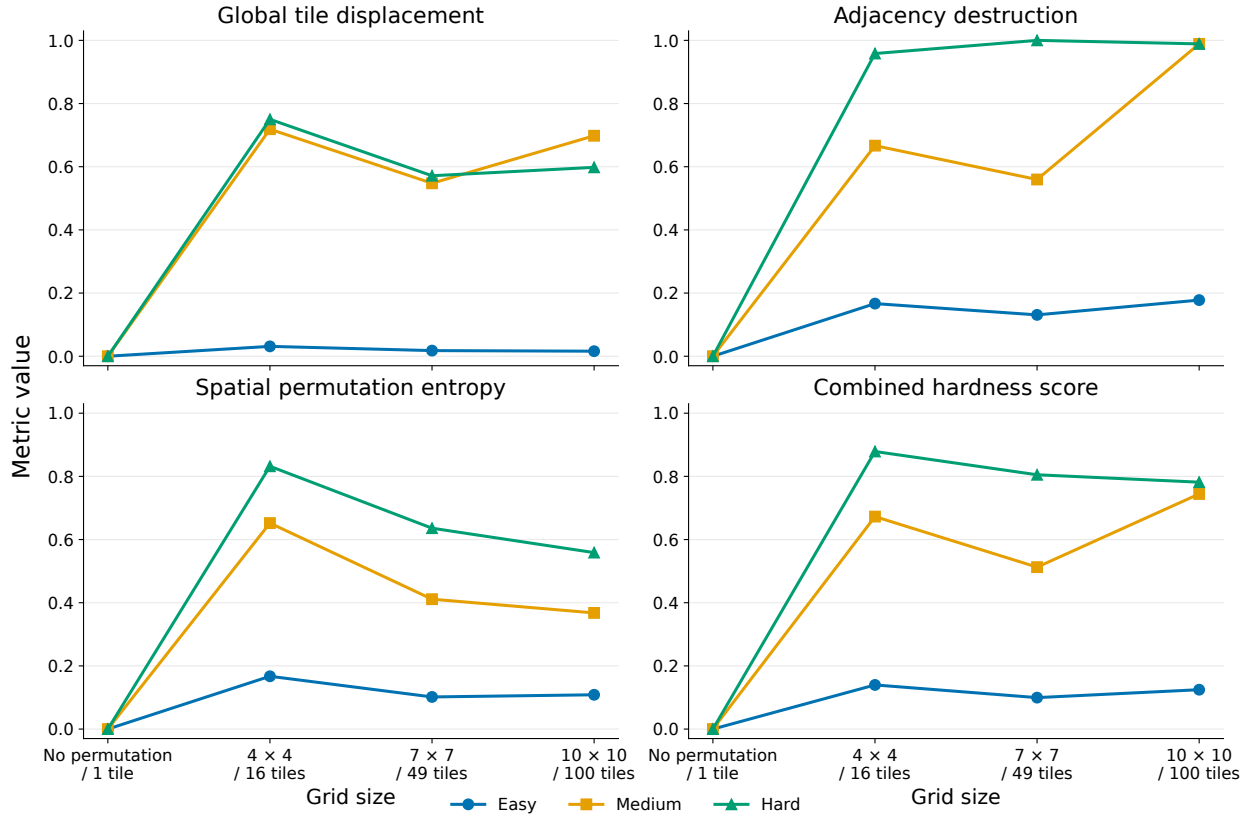


Figure 8: Part 3 hardness metrics by grid size and deterministic difficulty label. The four panels show global tile displacement, adjacency destruction, spatial permutation entropy, and the combined hardness score with grid size on the x-axis and metric value on the y-axis. These metrics describe the deterministic permutations and are model-agnostic. The plot is descriptive and does not imply statistical significance.

Item	Value to report
Dataset source	Kaggle Dogs vs Cats labeled training images; the supervised experiments use the labeled 25,000-image portion.
Train/validation split procedure	Stratified class-balanced split; validation fraction 0.2; test fraction 0.0; split seed 42.
Image preprocessing	Resize to the smallest square size divisible by the grid side length, convert to RGB float tensor, and normalize with ImageNet mean and standard deviation; timm models are center-cropped back to 224 after tile-compatible resizing when required.
Tile transform details	Tiles are square non-overlapping tensor blocks after resizing; the permutation copies source tiles into output positions with no interpolation during tile rearrangement; preset parameters are listed in Table 1.
Optimization	AdamW; learning rate 3×10^{-4} ; weight decay 1×10^{-4} ; cross-entropy loss; batch size 64 and 4 dataloader workers in the Colab runs; no learning-rate scheduler is used.
Training duration	Remote Part 1 runs train for 10 epochs; explicit Part 2 ablation runs train for 30 epochs; the zero-shot control uses 0 epochs.
Model configuration	Part 1 models use timm ImageNet-1k checkpoints: <code>mobilenetv3_small_100</code> , <code>deit_tiny_patch16_224.fb_in1k</code> , and <code>gmlp_s16_224.ra3_in1k</code> . The main Part 1 baseline freezes backbones and trains only binary heads; the controls either unfreeze the pretrained binary-head model or preserve the native ImageNet-1k head for zero-shot dog/cat scoring. The configurable-CNN MLP head is <code>Linear(in_features,256)</code> , <code>BatchNorm1d</code> , <code>ReLU</code> , <code>Dropout(0.3)</code> , <code>Linear(256,2)</code> .
Augmentation and curricula	Patch shuffle randomly permutes within-image patches during training; regular augmentation uses random rotation, color jitter, resize, tensor conversion, and normalization; mixed-input training uses $p_{\text{original}} = 0.5$; curriculum definitions are described in Section 4.6.
Compute environment	The executed environment records Python 3.12.13, numpy 2.0.2, pandas 2.2.2, scipy 1.16.3, scikit-learn 1.6.1, torch 2.11.0+cu128, torchvision 0.26.0+cu128, timm 1.0.27, PyYAML 6.0.3, and an NVIDIA L4 GPU. Seed 42 is used, but deterministic GPU kernels are not enforced.
Artifacts	The saved outputs include experiment scripts, raw and aggregated CSV files, joined hardness metrics, figures, and checkpoints for the reported run matrix.

Table 4: Reproducibility checklist for the reported experiment matrix.

Model	1 tile best	16 tiles best	49 tiles best	100 tiles best
MobileNetV3-Small	91.82%	86.93%	84.92%	80.36%
DeiT-Tiny	98.75%	97.23%	97.08%	92.97%
gMLP-S16	99.09%	97.16%	96.11%	89.47%

Table 5: Part 1 mean best validation accuracy by model and tile count, aggregated across the easy, medium, and hard deterministic records at each grid size.

Part 2 setting	Mean final acc.	Mean best acc.	Mean best delta vs. baseline
Regular Part 1 baseline	83.25%	83.48%	0.00 pp
Patch shuffle	84.99%	85.18%	+1.71 pp
Regular augmentations	87.93%	87.98%	+4.50 pp
Mixed original/permuted	87.75%	87.83%	+4.36 pp
CNN MLP head	91.57%	91.97%	+8.49 pp
Corruption-probability curriculum	88.29%	88.41%	+4.93 pp
Permutation-difficulty curriculum	88.30%	88.39%	+4.91 pp

Table 6: Part 2 MobileNetV3-Small CNN ablation summary from `part2_aggregated_results.csv`. “Mean final acc.” is the validation accuracy at the final epoch of the corresponding strategy. “Mean best acc.” is the best validation accuracy reached during that same strategy’s training run. Deltas are computed against the regular Part 1 reference baseline at the same grid size and deterministic difficulty label.

Model	Main operation	Inductive bias
MobileNetV3-Small	Convolution	Local texture and translation structure
DeiT-Tiny	Self-attention	Patch-to-patch interactions
gMLP-S16	Spatial-gating MLP	Patch and feature mixing without attention

Table 7: Architectural comparison of the three frozen feature extractors.

Model	Learnable	Frozen	Total
MobileNetV3-Small	2,050	1,517,856	1,519,906
DeiT-Tiny	386	5,524,416	5,524,802
gMLP-S16	514	19,165,656	19,166,170

Table 8: Parameter counts for the frozen-backbone comparison with two output classes. “Learnable” counts only the classifier head optimized during training; “Frozen” counts pre-trained parameters used only in the forward pass.

Model	Selected checkpoint	Native head	Freeze behavior
MobileNetV3-Small	mobilenetv3_small_100	classifier, 1000 classes	freeze all except <code>classifier</code>
DeiT-Tiny	deit_tiny_patch16_224.fb_in1k	head, 1000 classes	freeze all except <code>head</code>
gMLP-S16	gmlp_s16_224.ra3_in1k	head, 1000 classes	freeze all except <code>head</code>

Table 9: Pretraining, native classifier head, and freezing behavior for the selected Part 1 model checkpoints. All selected checkpoints are ImageNet-1k timm checkpoints; each native classifier exposes a 1000-class ImageNet-1k output head before the trained Dogs-versus-Cats binary head is attached.

Part 1 run	Training set	Validation set
1 tile / none	20,000 unpermuted images repeated under easy, medium, and hard labels.	5,000 unpermuted held-out images.
4×4 permutations	Same 20,000 images with easy, medium, and hard deterministic 4×4 permutations.	Same 5,000 held-out images with the matching deterministic permutation.
7×7 permutations	Same 20,000 images with easy, medium, and hard deterministic 7×7 permutations.	Same 5,000 held-out images with the matching deterministic permutation.
10×10 permutations	Same 20,000 images with easy, medium, and hard deterministic 10×10 permutations.	Same 5,000 held-out images with the matching deterministic permutation.

Table 10: Part 1 training and validation sets. The same split is reused for MobileNetV3-Small, DeiT-Tiny, and gMLP-S16, and each remote run trains for 10 epochs.